

Estacionamento Inteligente

Sprint 2 - MVP IoT

MQTT + HTTP + Postgres

Campus com 3 setores (A, B, C) e 30 vagas cada
90 sensores | 3 gateways | Backend reativo

Objetivo do projeto

MVP de estacionamento inteligente cobrindo as **7 etapas da prova**:

1. Simulacao de sensores e gateways
2. MQTT (eventos em tempo real)
3. Banco de dados (historico e relatorios)
4. API HTTP (consulta + relatorios)
5. Regra operacional (recomendacao quando setor lota)
6. Deteccao de incidentes (sensor travado / flapping)
7. Demonstracao ao vivo

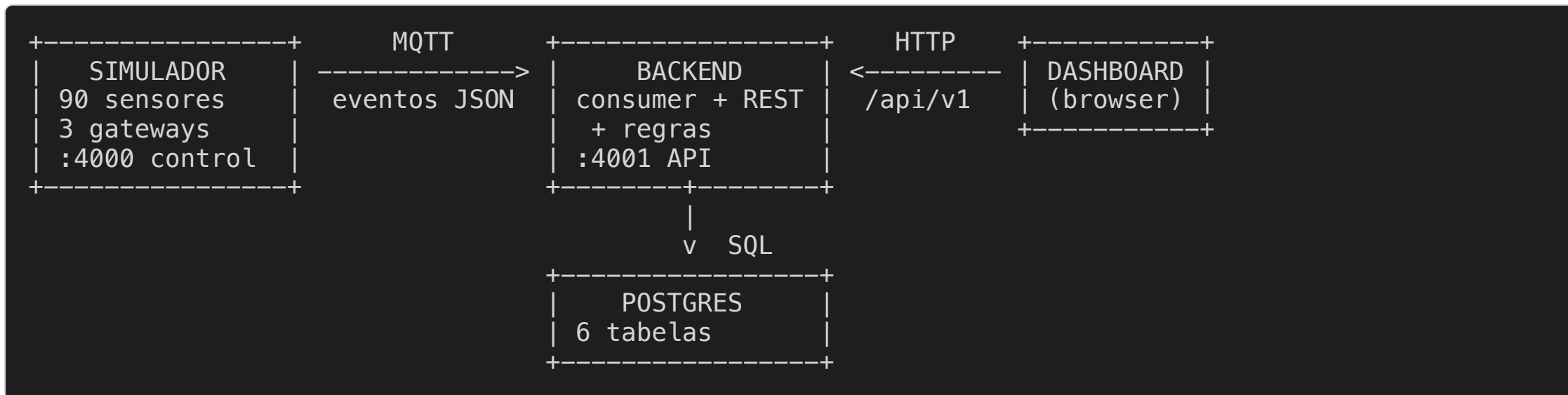
IA fica para a proxima sprint - aqui usamos so logica deterministica.

Stack tecnico

Camada	Tecnologia	Onde
Simulador	Node.js + mqtt	src/simulator/
Broker	Mosquitto (Docker)	docker-compose.yml
Backend	Node.js + Express + pg	src/backend/
Banco	PostgreSQL 16 (Docker)	configs/schema.sql
Dashboard	HTML + JS vanilla	public/

```
docker compose up -d mosquitto postgres
npm run backend      # terminal 1
npm start            # terminal 2 (simulador)
# abrir http://localhost:4001/
```

Arquitetura



- Simulador publica eventos no broker.
- Backend consome, valida, persiste, aplica regras, expoe REST.
- Dashboard faz polling de 5s na REST e desenha o mapa.

Estrutura de pastas

```

Prova-Sprint2-main/
|-- docker-compose.yml      Mosquitto + Postgres
|-- configs/schema.sql      Tabelas Postgres (Etapa 3)
|-- src/
|   |-- config.js           Constantes + topicos MQTT
|   |-- simulator/         ETAPA 1
|       |-- index.js       Entry: 90 sensores + 3 gateways
|       |-- sensor.js       State machine de 1 vaga
|       |-- gateway.js      Cliente MQTT por setor + LWT
|       |-- controlServer.js HTTP :4000 pra injetar falha
|       |-- layout.js       Gera A-01..C-30
|   |-- backend/           ETAPAS 2-6
|       |-- index.js       Subscriber MQTT + roteamento
|       |-- db.js          Persistencia + queries de leitura
|       |-- httpApi.js     API HTTP unificada (:4001)
|       |-- recommendations.js R-OP1 (>= 90%)
|       |-- incidents.js   Detector FLAPPING + scanner STUCK
|       |-- incidentDb.js  Storage de incidentes
|-- public/                Dashboard estatico (extra)

```

Etapa 1

Cenario e simulacao

Etapa 1: o que a prova exige

- Layout fixo: setores **A, B, C** com **30 vagas** cada (A-01..C-30)
- Simular **90 sensores + 3 gateways** em Node.js
- Padroes realistas:
 - horarios de pico (manha e fim de tarde)
 - permanencia 30min - 6h em tempo simulado
- Modo de teste pra injetar falhas:
 - `stuck_occupied`, `stuck_free`, `flapping`
- Tempo simulado configuravel (1s real = 1min sim, etc.)

Etapa 1: layout fixo

Arquivos: `src/simulator/layout.js`, `src/config.js`

```
// src/config.js
sectors: ['A', 'B', 'C'],
spotsPerSector: 30,
```

```
// src/simulator/layout.js
function listAllSpots() {
  const list = [];
  for (const sectorId of ['A', 'B', 'C']) {
    for (let i = 1; i <= 30; i++) {
      list.push({ sectorId, spotId: `${sectorId}-${String(i).padStart(2, '0')}` });
    }
  }
  return list;
}
```

`src/simulator/index.js` consome `listAllSpots()` e cria 1 `Sensor` por vaga.

Etapa 1: 90 sensores + 3 gateways

Arquivos: `sensor.js`, `gateway.js`, `index.js`

- **`sensor.js`** - 1 instancia por vaga. Mantem `state` (FREE/OCCUPIED), `dwell` (tempo restante), `fault` (falha injetada). Cada tick decide se troca de estado.
- **`gateway.js`** - 1 por setor. Cada gateway e **seu proprio cliente MQTT** (modela 3 dispositivos fisicos no campus). Configura **LWT** (Last Will): se cair sem aviso, broker publica `{status:"offline"}` sozinho.
- **`index.js`** - cria 90 sensores + 3 gateways, espera os 3 conectarem, publica snapshot inicial das 90 vagas e roda loop principal.

```
// src/simulator/index.js
for (const { sectorId, spotId } of listAllSpots()) {
  sensorsByKey[spotId] = new Sensor({ sectorId, spotId });
}
for (const sectorId of config.sectors) {
  gateways[sectorId] = new Gateway({ sectorId, sensors: sensorsBySector[sectorId] });
}
```

Etapa 1: padroes realistas

Arquivo: `src/simulator/sensor.js`

```
function probabilidadeChegada(hora) {  
  if (hora >= 7 && hora < 9) return 0.06; // pico manha  
  if (hora >= 11 && hora < 14) return 0.04; // almoco  
  if (hora >= 17 && hora < 19) return 0.07; // pico fim de tarde  
  if (hora >= 19 && hora < 22) return 0.015;  
  return 0.003; // madrugada  
}  
  
function escolherTempoPermanencia() {  
  const r = Math.random();  
  if (r < 0.55) return 30 + Math.floor(Math.random() * 60); // 30-90 min  
  if (r < 0.85) return 90 + Math.floor(Math.random() * 90); // 90-180 min  
  return 180 + Math.floor(Math.random() * 180); // 180-360 min  
}
```

A "hora do dia" vem de um **relogio simulado** em `index.js` que avanca 1 min sim por tick (`TICK_MS` ms reais).

Etapa 1: injeção de falhas

Arquivo: `src/simulator/controlServer.js` (HTTP:4000)

```
# injetar
curl -X POST http://localhost:4000/faults \
  -H 'Content-Type: application/json' \
  -d '{"spotId":"A-07","fault":"stuck_occupied"}'

# limpar
curl -X DELETE http://localhost:4000/faults
```

Tipos: `stuck_occupied`, `stuck_free`, `flapping`. A lógica de cada um está em `sensor.js`:

```
if (this.fault === 'stuck_occupied') this.state = OCCUPIED;
else if (this.fault === 'stuck_free') this.state = FREE;
else if (this.fault === 'flapping') {
  this.flapCounter++;
  if (this.flapCounter >= this.flapEvery) {
    this.state = this.state === FREE ? OCCUPIED : FREE;
    this.flapCounter = 0;
  }
}
```

Etapa 2

MQTT - topicos e ingestao

Etapa 2: topicos obrigatorios

Definidos em `src/config.js` (linhas 51-66)

```
campus/parking/sectors/<sectorId>/spots/<spotId>/events
campus/parking/sectors/<sectorId>/gateway/status
campus/parking/recommendations           (extra, publicado pelo backend)
```

Topico	Publisher	Quando
<code>.../events</code>	<code>gateway.publishSpotEvent</code>	Toda mudanca de estado
<code>.../gateway/status</code>	<code>gateway.publishStatus</code>	Heartbeat 15s + LWT
<code>.../recommendations</code>	<code>recommendationEngine</code>	Setor cruza 90%

Etapa 2: payload do evento

Montado em `src/simulator/gateway.js:69-82`

```
{
  "eventId": "uuid v4",
  "ts": "2026-05-06T19:15:30.000Z",
  "sectorId": "A",
  "spotId": "A-07",
  "state": "OCCUPIED",
  "source": "sensor"
}
```

Backend valida em `src/backend/index.js:48-68`:

- Todos os campos obrigatorios presentes
- `state in {FREE, OCCUPIED}`
- Topico bate com `sectorId / spotId` do payload

Etapa 2: idempotencia por eventId

Arquivo: `src/backend/db.js:24-42`

```
INSERT INTO spot_events (event_id, ts, sector_id, spot_id, state, source, raw_payload)
VALUES ($1, $2, $3, $4, $5, $6, $7)
ON CONFLICT (event_id) DO NOTHING
RETURNING event_id
```

- Se `RETURNING` devolve linha = inseriu agora (novo evento)
- Se nao devolve = duplicado (broker reentregou via QoS 1)

```
// src/backend/index.js
const inserido = await db.saveEvent(evt);
if (inserido) {
  await db.updateSpotState(evt); // atualiza estado atual
  await incidentDetector.processar(evt); // detector de incidentes
  await recommendationEngine.processSpotChange(evt.sectorId); // R-OP1
}
```

Logs do backend mostram `inseridos=232 duplicados=15` em tempo real.

Etapa 3

Banco de dados (Postgres)

Etapa 3: tabelas exigidas

Schema completo em `configs/schema.sql` (auto-aplicado pelo Postgres)

Tabela	Conteudo	Linhas
<code>spots</code>	Estado atual de cada vaga (upsert)	15-25
<code>spot_events</code>	Historico bruto, PK em <code>event_id</code>	30-45
<code>sector_snapshots</code>	Snapshot agregado por minuto	50-60
<code>incidents</code>	Anomalias detectadas	65-86
<code>recommendations_log</code>	Log da R-OP1	91-101
<code>gateway_status</code>	Aux: ultimo status conhecido	106-111

```
# docker-compose.yml monta o schema em initdb.d
postgres:
  volumes:
    - ./configs/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql:ro
```

Etapa 3: indice unico parcial em incidents

```
CREATE TABLE incidents (  
  id          UUID PRIMARY KEY,  
  ts_open     TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  ts_close    TIMESTAMPTZ,  
  type        TEXT NOT NULL CHECK (type IN ('STUCK_OCCUPIED', 'STUCK_FREE', 'FLAPPING')),  
  severity    TEXT NOT NULL,  
  sector_id   TEXT NOT NULL,  
  spot_id     TEXT,  
  evidence_json JSONB NOT NULL DEFAULT '{}',  
  status       TEXT NOT NULL DEFAULT 'open' CHECK (status IN ('open', 'closed')),  
  last_seen_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
-- So pode existir UM incidente aberto por (spot, tipo) ao mesmo tempo  
CREATE UNIQUE INDEX incidents_open_unique  
  ON incidents (spot_id, type)  
  WHERE status = 'open';
```

Quando o incidente fecha, sai do indice -> um novo pode ser aberto.

Etapa 3: acesso ao banco no código

Arquivo	Funcao
src/backend/pool.js	Pool unico do node-postgres + waitForReady() no boot
src/backend/db.js	Eventos, spots, mapa, setores, turnover, snapshots, recomendacoes
src/backend/incidentDb.js	Queries de incidentes (com transacao)
src/backend/snapshotter.js	Job que insere sector_snapshots a cada 60s

```
// src/backend/pool.js - retry no boot
async function waitForReady({ maxAttempts = 30, delayMs = 1000 } = {}) {
  for (let i = 1; i <= maxAttempts; i++) {
    try { await pool.query('SELECT 1'); return; }
    catch { await new Promise(r => setTimeout(r, delayMs)); }
  }
}
```

Etapa 4

API HTTP - consulta e relatorios

Etapa 4: endpoints de consulta

Arquivo: `src/backend/httpApi.js` (porta :4001)

Endpoint	Linha	Implementacao
<code>GET /api/v1/map</code>	43-48	<code>db.getMap()</code>
<code>GET /api/v1/sectors</code>	50-58	<code>db.getSectorsSummary()</code>
<code>GET /api/v1/sectors/:id/spots</code>	60-74	<code>db.getSectorSpots()</code>
<code>GET /api/v1/sectors/:id/free-spots?limit=N</code>	76-96	<code>db.getFreeSpots()</code>

```
curl http://localhost:4001/api/v1/sectors
# [{"sectorId":"A","occupiedCount":24,"freeCount":6,"occupancyRate":0.8,...}]

curl http://localhost:4001/api/v1/sectors/A/free-spots?limit=5
```

Etapa 4: relatorio de turnover

Endpoint em `httpApi.js:102-123`, query em `db.js:198-234`

```
WITH transitions AS (  
  SELECT sector_id, state, ts,  
         LAG(state) OVER (PARTITION BY spot_id ORDER BY ts) AS prev_state  
  FROM spot_events  
)  
SELECT sector_id, COUNT(*)::int AS turnover  
FROM transitions  
WHERE prev_state = 'FREE' AND state = 'OCCUPIED'  
      AND ($1::text IS NULL OR sector_id = $1)  
      AND ($2::timestampz IS NULL OR ts >= $2)  
      AND ($3::timestampz IS NULL OR ts <= $3)  
GROUP BY sector_id;
```

`turnover` = numero de transicoes **FREE -> OCCUPIED** = veiculos atendidos.

```
curl 'http://localhost:4001/api/v1/reports/turnover?sectorId=A&from=2026-05-06T00:00Z'
```

Etapa 5

Regra operacional R-OP1

Etapa 5: regra R-OP1 ($\geq 90\%$)

Arquivo: `src/backend/recommendations.js`

Apos cada update de estado, o backend chama `processSpotChange(sectoId)`:

1. Le ocupacao atual do setor
2. Detecta **borda de subida** (cruzou 90%) - so dispara uma vez por subida
3. Calcula recomendacao com `compute(fromSector)`
4. Persiste em `recommendations_log`
5. Publica em MQTT no topico `campus/parking/recommendations`

```
// recommendations.js:48-53
const candidatos = outros
  .map((s, i) => ({ sectorId: s, ...stats[i] }))
  .filter((c) => c.freeCount > 0 && c.occupancyRate < 0.9)
  .sort((a, b) => b.freeCount - a.freeCount // mais livres primeiro
        || a.sectorId.localeCompare(b.sectorId));
const escolhido = candidatos[0];
```


Etapa 5: saida JSON

Endpoint on-demand em `httpApi.js:129-147`

```
curl 'http://localhost:4001/api/v1/recommendation?fromSector=A'
```

```
{
  "fromSector": "A",
  "recommendedSector": "B",
  "reason": "Sector A at 93% occupancy; Sector B has 12 free spots",
  "ts": "2026-05-06T10:20:00.000Z",
  "fromOccupancyRate": 0.933,
  "candidates": [
    {"sectorId": "B", "freeCount": 12, "occupancyRate": 0.6},
    {"sectorId": "C", "freeCount": 8, "occupancyRate": 0.733}
  ]
}
```

Demo: `POST /faults` com `sectorId:"A", fault:"stuck_occupied"` -> setor A sobe pra 100% -> recomendacao gerada automaticamente.

Etapa 6

Deteccao de incidentes

Etapa 6: dois caminhos de deteccao

Arquivos: `src/backend/incidents.js` + `incidentDb.js` + `incidentApi.js`

Por que dois caminhos? Um sensor travado **nao publica novos eventos** - o detector orientado a evento sozinho nao pegaria.

Tipo	Detectado por	Latencia
FLAPPING	Caminho 1: orientado a evento (a cada <code>processar(evt)</code>)	~1-2s
STUCK_OCCUPIED	Caminho 2: scanner periodico (varre tabela <code>spots</code>)	~48s
STUCK_FREE	Caminho 2: scanner periodico	~48s

Indice unico parcial em `incidents (spot_id, type) WHERE status='open'` impede duplicar.

Etapa 6: caminho 1 - FLAPPING (online)

`incidents.js:43-110` - chamado a cada evento ingerido

Mantem em memoria por vaga: `lastState`, `lastChangeTs`, `changesInWindow[]` (timestamps das trocas dentro de uma janela rolante).

```
async processar(evt) {
  const h = this.historico.get(spotId);
  if (state === h.lastState) return; // sem mudanca, sai

  h.changesInWindow.push(agora);
  h.changesInWindow = h.changesInWindow.filter(
    (t) => agora - t <= TH.flappingWindowMs
  );

  // Toda transicao recupera STUCK (sensor voltou a funcionar)
  await this._fecharSeExistir(spotId, 'STUCK_OCCUPIED');
  await this._fecharSeExistir(spotId, 'STUCK_FREE');

  if (h.changesInWindow.length > TH.flappingMaxChanges) {
    await incidentDb.openIncident({ type: 'FLAPPING', ... });
  }
}
```

Etapa 6: caminho 2 - STUCK (scanner)

incidents.js:115-156 - roda a cada STUCK_SCAN_INTERVAL_MS

```
SELECT spot_id, sector_id, state, last_ts,  
       EXTRACT(EPOCH FROM (NOW() - last_ts)) AS seconds_in_state  
FROM spots  
WHERE last_ts IS NOT NULL  
      AND last_ts < NOW() - ($1::bigint * INTERVAL '1 millisecond');
```

Para cada linha:

- state = 'OCCUPIED' -> abre STUCK_OCCUPIED
- state = 'FREE' -> abre STUCK_FREE

```
// src/backend/index.js - plug do scanner  
startSnapshotter();  
incidentDetector.startStuckScanner();
```

Etapa 6: persistencia transacional

incidentDb.js:48-94 - BEGIN + SELECT FOR UPDATE

```
async function openIncident({ type, sectorId, spotId, evidenceJson }) {  
  const client = await pool.connect();  
  try {  
    await client.query('BEGIN');  
    const existing = await client.query(  
      `SELECT * FROM incidents  
      WHERE spot_id=$1 AND type=$2 AND status='open' FOR UPDATE`,  
      [spotId, type]);  
  
    if (existing.rowCount > 0) {  
      // Merge JSONB evidence + atualiza last_seen_at  
      await client.query(`UPDATE incidents  
        SET evidence_json = evidence_json || $2::jsonb, last_seen_at = NOW()  
        WHERE id = $1`, [existing.rows[0].id, evidenceJson]);  
    } else {  
      await client.query(`INSERT INTO incidents (...) VALUES (...)`);  
    }  
    await client.query('COMMIT');  
  } catch (e) { await client.query('ROLLBACK'); throw e; }  
}
```

Etapa 6: calibracao TICK_MS x thresholds

Threshold de STUCK precisa ser **estritamente maior** que o dwell maximo (6h sim), senao saudaveis viram STUCK.

TICK_MS	STUCK_THRESHOLD_MS	FLAPPING_WINDOW_MS	Demo
100 (10x rapido)	48000	6000	~48s
200 (5x rapido)	96000	12000	~96s
1000 (real)	480000	60000	~8min

Em todas as linhas o spec e respeitado em **tempo simulado**: STUCK 8h, FLAPPING > 6 trocas em 60min, dwell maximo 6h.

Default deste repo: `TICK_MS=100` , demo de STUCK em ~48s.

Etapa 6: API de incidentes

src/backend/incidentApi.js

```
# todos abertos
curl 'http://localhost:4001/api/v1/incidents?status=open'

# filtros
curl 'http://localhost:4001/api/v1/incidents?type=FLAPPING'
curl 'http://localhost:4001/api/v1/incidents?sectorId=A'
curl 'http://localhost:4001/api/v1/incidents?spotId=A-07'

# stats
curl 'http://localhost:4001/api/v1/incidents/stats'
# {"total":12,"open":3,"closed":9,"byType":{"STUCK_OCCUPIED":1,"STUCK_FREE":1,"FLAPPING":1}}
```

No dashboard, os 3 tipos aparecem no painel "Incidentes abertos" e as vagas afetadas viram laranja no mapa.

Etapa 7

Demonstracao

Etapa 7: checklist da prova

Item	Como mostrar
Subir Mosquitto + servicos + banco	<code>docker compose up -d</code> + <code>npm run backend</code> + <code>npm start</code>
Simulador gerando eventos MQTT	Logs do <code>npm start</code> , ou inspecionar broker
<code>/map</code> e <code>/sectors</code> em tempo real	Dashboard ou <code>curl /api/v1/map</code>
Injetar falha e ver incidente	<code>POST :4000/faults</code> + <code>GET :4001/api/v1/incidents</code>
Lotar setor e ver recomendacao	<code>POST :4000/faults</code> setorial + <code>GET :4001/api/v1/recommendation</code>

Roteiro da demo (script)

```
# 1) sobe tudo
docker compose up -d mosquitto postgres
npm run backend # T1
npm start      # T2

# 2) abrir dashboard em http://localhost:4001/
#     mostrar mapa 3x30, setores, gateways online

# 3) injetar 3 falhas (uma de cada tipo)
curl -X POST http://localhost:4000/faults -H 'Content-Type: application/json' \
  -d '{"spotId":"A-07","fault":"stuck_occupied"}'
curl -X POST http://localhost:4000/faults -H 'Content-Type: application/json' \
  -d '{"spotId":"B-12","fault":"stuck_free"}'
curl -X POST http://localhost:4000/faults -H 'Content-Type: application/json' \
  -d '{"spotId":"C-03","fault":"flapping"}'

# 4) lotar setor A pra disparar recomendacao
curl -X POST http://localhost:4000/faults -H 'Content-Type: application/json' \
  -d '{"sectorId":"A","fault":"stuck_occupied"}'

# 5) inspecionar
curl http://localhost:4001/api/v1/incidents?status=open
curl http://localhost:4001/api/v1/recommendation?fromSector=A
```

Resumo: o que cada etapa cobre

Etapa	Pasta principal	Funcao
1	src/simulator/	Gera eventos MQTT realistas + falhas
2	src/backend/index.js + db.js	Consome MQTT, valida, idempotencia
3	configs/schema.sql + pool.js + snapshotter.js	6 tabelas Postgres
4	httpApi.js	Mapa, setores, free-spots, turnover
5	recommendations.js	Regra R-OP1 (>= 90%)
6	incidents.js + incidentDb.js + incidentApi.js	FLAPPING (online) + STUCK (scanner)
7	demo ao vivo	Checklist completo

Obrigado

Perguntas?

Codigo: Prova-Sprint2-main/

Dashboard: <http://localhost:4001/>

Guia tecnico completo: [GUIA-PROVA.md](#)